

2025 年编译原理与技术实践课程实验报告

姓名：张建夫

学号：10235101477

课程名称：编译原理与技术实践

指导教师：张敏

实验日期：2025 年 9 月 15 日 ~ 2026 年 1 月 9 号

实践亮点

以下为本课程实践环节中的主要亮点（不超过十项）：

1. 在词法分析器项目中，设计了 `ErrorReporter` 类，用于存储在词法分析过程中的所有错误与警告，并且能够检测出几种常见的词法错误（`error_reporter.h` 中 `ctrl+F` 搜索 “`ErrorMessages`” 查看所有支持的词法错误），具体的检测代码搜索 “`reporter_>Report`”，可以看到所有的错误处理。
2. 在词法分析器项目中，考虑了单引号所包围的变量为常量（包括转义符）的情况（`c_keys.txt` 中没有单引号），具体代码在 `lexical_analyzer.cpp` 中，搜索 “`SingleQuotationMark`”，使词法分析器功能更为强大。
3. 在词法分析器项目中，使用面向对象设计，存储 token 的实现了一个类 `TokenStream`，存储错误以及报警的实现一个类 `ErrorReporter`，编译器内部错误实现了一个类 `LexerException`，符号表（存储关键字和所有标识符）实现了一个类 `SymbolTable`（使用哈希表加速查找），这四个均为词法器类 `LexicalAnalyzer` 服务，遵循了面向对象设计中的单一职责原则和开闭原则，具体的代码可以查看相应的头文件与源文件（或者搜索上面出现的类名）。
4. 在词法分析器项目中，撰写了额外的测试用例，实现了更全面的功能检测，具体测试用例以及测试的功能见 `test_data/project1` 下以 `my_test` 开头的测试文件。
5. 在 LL1 语法分析器中，实现了分析表的自动生成，具体代码见 `ll1_analyzer.cpp` 中的 `BuildTable` 方法，并撰写了测试用例，具体测试方式看 `readme.md`
6. 在 LL1 语法分析器和 LR1 语法分析器中，复用词法分析器项目中的所有代码（错误处理的类仍然使用 `ErrorReporter`，测试用例的分词使用的是之前的词法分析器），使项目各部分相关联，形成 词法->语法 的连贯编译过程。
7. 在 LL1 语法分析器中，实现了语法树的抽象化（即删除没有语义的节点，如 `a = b + c;`，语法树化简后（没有哪些 `stmt` 之类的非终结符了）变为：

=

a

+

b

c

一个缩进表示一个层级，“=” 表示整个式子，而 “+” 号表示 “`b + c`”。

8. 在 LR1 语法分析器中，实现了从原始的产生式字符串到 `action table` 和 `goto table` 的全自动构建，可以查看 `slr1_analyzer.cpp` 中的 `ComputeFirst`, `CreateSLRTable` 等方法。
9. 在 LR1 语法分析器中，实现了 `action table` 和 `goto table` 的可视化生成。
10. 在项目管理中使用 `Git` 进行版本控制，实现了代码变更的透明化追踪，github 仓库：
<https://github.com/saydontgo/ECNU-compiler-project>。